

Introduction au microcontrôleur PIC16F84A

Polo Antoine

1. INTRODUCTION

Les PIC sont une famille de microcontrôleurs 8 bits fabriqués par Microchip Technology. Le PIC16F84A est un microcontrôleur populaire de cette famille, connu pour sa simplicité et sa polyvalence. Il est largement utilisé dans les projets électroniques et les applications embarquées.

Au fil des années, il a construit une solide réputation dans le milieu industriel et éducatif, bien qu'aujourd'hui largement dépassé par les microcontrôleurs précédemment abordés comme les STM32 ou les puces AVR.

2. FICHE TECHNIQUE

- **Architecture RISC** : Set de 35 instructions seulement, exécutées en 1 cycle (sauf sauts).
- **Mémoire Flash** : 1K octets.
- **RAM** : 68 octets.
- **EEPROM** : 64 octets d'EEPROM.
- **Broches d'E/S** : 13 E/S.
- **Timers** : 1 timer de 8 bits.
- **ADC/DAC** : Aucun.
- **UART/SPI/I2C** : Aucun.
- **Oscillateur** : 20 MHz (max).
- **Tension d'alimentation** : 2V à 5.5V.
- **Température de fonctionnement** : -40°C à +85°C
- **Boîtier** : DIP-18, SOIC-20.

Cette architecture très minimaliste lui donne sa réputation dans le milieu éducatif ; chaque protocole doit être codé par l'utilisateur, chaque périphérique doit être implémenté dans le circuit. Cela permet une compréhension plus profonde du fonctionnement d'un microcontrôleur et de ses interactions avec les composants externes.

3. PROGRAMMATION

Contrairement aux cartes Arduino, le PIC16F84A ne possède pas d'interface de programmation intégrée. Pour programmer le microcontrôleur, vous aurez besoin d'un programmeur externe compatible avec les microcontrôleurs PIC. Dans notre cas, nous allons utiliser le programmeur PICKIT 5 de Microchip que nous pouvons voir à la figure suivante.



Figure 1. Programmeur PICKIT 5 de Microchip.

Plusieurs logiciels existent pour programmer le PIC16F84A, mais nous allons utiliser MPLAB X IDE, un environnement de développement intégré fourni par Microchip. Il offre une interface conviviale pour écrire, compiler et télécharger le code sur le microcontrôleur. Nous utiliserons également le compilateur XC8, avec le langage C.

Nous allons le voir, la programmation de ces microcontrôleurs est très différente de celle des cartes Arduino, mais elle offre une flexibilité et un contrôle plus précis sur le matériel. Aucune bibliothèque

n'est disponible, il faudra donc coder chaque fonctionnalité que nous voulons implémenter dans notre projet.

4. CIRCUIT

Nous allons dans un premier temps faire un simple exemple de blink. Pour cela, nous allons utiliser le circuit suivant, que nous pouvons voir à la figure suivante. Pour cela, il nous faudra une LED ainsi que sa résistance, une résistance de 10kΩ, deux condensateurs de 22pF ainsi qu'un quartz de 8MHz.

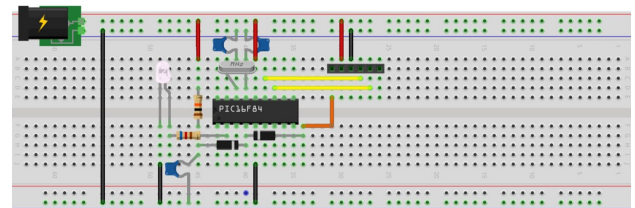


Figure 2. Circuit de base pour un exemple de blink.

Note

Il est également possible d'ajouter un condensateur de 100nF ainsi que deux diodes de protection, mais cela reste optionnel.

5. CRÉATION DU PROJET

Une fois MPLAB X IDE installé, nous allons créer un nouveau projet. Pour cela, nous allons cliquer sur "File" puis "New Project". Nous allons ensuite sélectionner "Microchip Embedded" puis "Standalone Project". Nous pouvons ensuite sélectionner le microcontrôleur PIC16F84A, puis le compilateur XC8. Nous allons ensuite donner un nom à notre projet et choisir un emplacement pour le sauvegarder.

6. PROGRAMME

Nous ne allons pour le moment pas nous attarder sur le programme, nous allons simplement nous contenter de faire clignoter la LED.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRT = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

void main(void) {
    TRISAbits.TRISA3 = 0;

    PORTAbits.RA3 = 0;

    while(1) {
        PORTAbits.RA3 = 1;
        __delay_ms(500);

        PORTAbits.RA3 = 0;
        __delay_ms(500);
    }
}
```

Code 1. Programme blink.

Afin de placer ce programme, il faudra créer un nouveau fichier source. Pour cela, nous allons faire un clic droit sur le dossier "Source Files" puis "New" et "C Source File". Nous allons ensuite donner un nom à notre fichier, puis cliquer sur "Finish".

7. ENVOI DU CODE

Une fois le programme écrit, nous allons le compiler en cliquant sur l'icône "Build Project" (ou en appuyant sur F11). Si tout se passe bien, nous devrions voir un message indiquant que la compilation s'est terminée avec succès.

Nous allons ensuite nous rendre dans les paramètres du projet en cliquant sur "File" puis "Project Properties". Nous allons ensuite sélectionner "PICkit 5" dans la section "Hardware Tool", puis cliquer sur "Apply". Une catégorie PICkit 5 va apparaître ; dans cette dernière, nous allons nous rendre dans la catégorie "Power" et sélectionner "Power target circuit from PICkit 5". Nous allons ensuite cliquer sur "OK".

Nous allons ensuite cliquer sur l'icône "Make and Program Device Main Project" (ou en appuyant sur Shift + F11). Le programmeur va alors envoyer le code sur le microcontrôleur. Si tout se passe bien, nous devrions voir un message indiquant que la programmation s'est terminée avec succès.

Si tout se passe bien, la LED devrait maintenant clignoter à une fréquence de 1Hz, félicitations ! Si ce n'est pas le cas, vérifiez les branchements du circuit ainsi que les paramètres du projet.

8. ANALYSE DU CODE

Contrairement à l'écosystème Arduino qui masque la gestion du matériel derrière des fonctions de haut niveau, la programmation en C avec le compilateur XC8 impose de manipuler directement les registres et la configuration matérielle de la puce. Analysons les éléments clés de notre programme.

8.1. Les bits de configuration (Fuses)

Avant même le démarrage du programme, les directives `#pragma config` définissent le comportement matériel de base du microcontrôleur :

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config CP = OFF
```

- **FOSC = HS** : Configure l'oscillateur en mode *High Speed*. Ce mode est indispensable car nous utilisons un quartz externe de 8 MHz (supérieur à 4 MHz).
- **WDTE = OFF** : Désactive le *Watchdog Timer*. S'il était activé, ce compteur matériel réinitialiserait le PIC en boucle si le code ne venait pas le vider régulièrement, bloquant notre clignotement.
- **PWRTE = OFF** : Désactive le *Power-up Timer*, un délai de sécurité au démarrage lié à la stabilisation de la tension.
- **CP = OFF** : Désactive la protection du code (*Code Protection*), permettant de relire la mémoire Flash du PIC après programmation.

8.2. L'inclusion de xc.h

La ligne `#include <xc.h>` est fondamentale. Il s'agit de la bibliothèque maîtresse du compilateur XC8. C'est elle qui fait automatiquement le lien entre le modèle de PIC sélectionné dans le projet

(ici, le PIC16F84A) et les noms des registres dans le code. Sans elle, le compilateur ne comprendrait pas ce que signifient des termes comme `TRISAbits` ou `PORTA`.

8.3. La configuration de la fréquence

La macro suivante est indispensable pour la gestion du temps :

```
#define _XTAL_FREQ 8000000
```

Elle indique au compilateur que la fréquence de notre quartz est de 8 MHz. Cela permet aux macros internes comme `__delay_ms()` de calculer précisément le nombre exact de cycles d'horloge à consommer pour suspendre l'exécution durant 500 ms.

8.4. Les registres TRIS et PORT

L'accès aux broches d'Entrées/Sorties s'effectue via deux registres :

- **TRISA** : Configure la direction des broches du PORTA. Le bit `TRISA3 = 0` configure la broche **RA3** en sortie (*0 = Output*).
- **PORTA** : Contrôle l'état logique de la broche. `PORTAbits.RA3 = 0` force initialement la broche à l'état bas (0V) avant d'entrer dans la boucle.

8.5. La boucle principale de clignotement

À l'intérieur de la boucle infinie `while(1)`, l'allumage et l'extinction de la LED sont gérés de manière séquentielle :

```
PORTAbits.RA3 = 1;
__delay_ms(500);

PORTAbits.RA3 = 0;
__delay_ms(500);
```

Le programme applique d'abord un état haut (5V) sur RA3 pour allumer la LED, attend 500 ms, puis applique un état bas (0V) pour l'éteindre pendant à nouveau 500 ms. Ce cycle se répète indéfiniment, créant un clignotement régulier à une fréquence de 1 Hz.

9. CONCLUSION

Bien que le PIC16F84A soit aujourd'hui technologiquement dépassé par des architectures plus modernes comme les puces AVR ou les microcontrôleurs STM32, il n'en demeure pas moins un outil pédagogique d'une valeur inestimable. Son architecture RISC épurée et l'absence totale de périphériques matériels complexes obligent le développeur à s'approprier les bases fondamentales des systèmes embarqués : la manipulation directe des registres, la gestion rigoureuse de la mémoire et la configuration matérielle par les *fuses*.

En maîtrisant la mise en œuvre de ce composant historique depuis le câblage de son horloge externe jusqu'à l'écriture de sa boucle de contrôle en C, on acquiert une compréhension profonde et transparente des interactions entre le logiciel et le matériel. Cette base solide constitue un tremplin idéal pour aborder, par la suite, des architectures plus avancées avec une vision beaucoup plus claire de ce qui se passe réellement sous le capot de nos lignes de code.